COL 1101

LECTURE 10

AUGUST 13, 2026

STACKS & QUEUES

LAST CLASS : THE STACK ADT

- data can be read/deleted / inserted at one end of the list

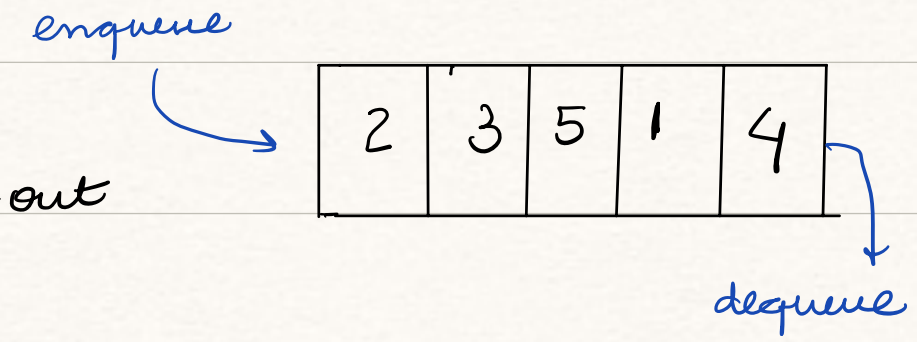
Operations Last-In-First-Out

- push(x) : put x on the top
- pop() : remove & return the top
- peek() : return the top
- size() : return the # of elements



Two implementations : ARRAYSTACK & LINKEDSTACK

TODAY'S CLASS : THE QUEUE ADT



Queue follows First-in, first-out discipline

Operations

→ Enqueue (e) : Insert element e to the back of queue

→ Dequeue () : Remove & return the first element from queue

EXAMPLE

Enqueue (10) [10]

Enqueue (50) back ↖ [50, 10] → front

Enqueue (45) [45, 50, 10]

size ( ) 3

Dequeue ( ) back ↖ [45, 50] ↗ front return 10

Dequeue ( ) [45] return 50

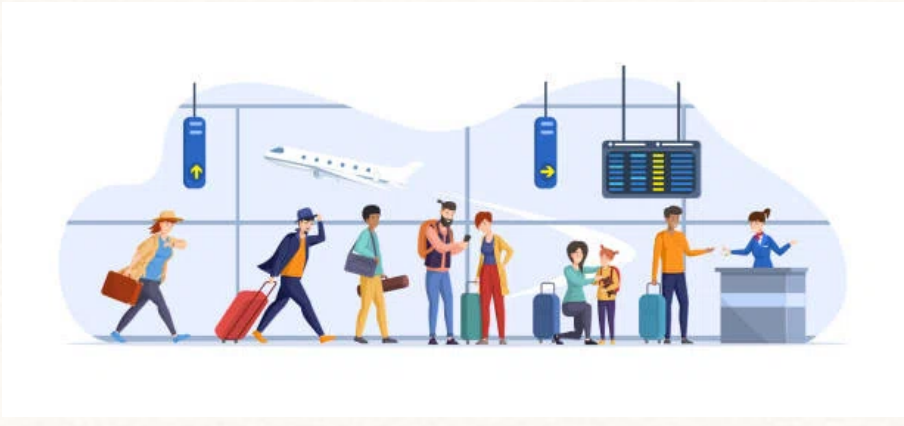
WHY STUDY THE QUEUE ADT?

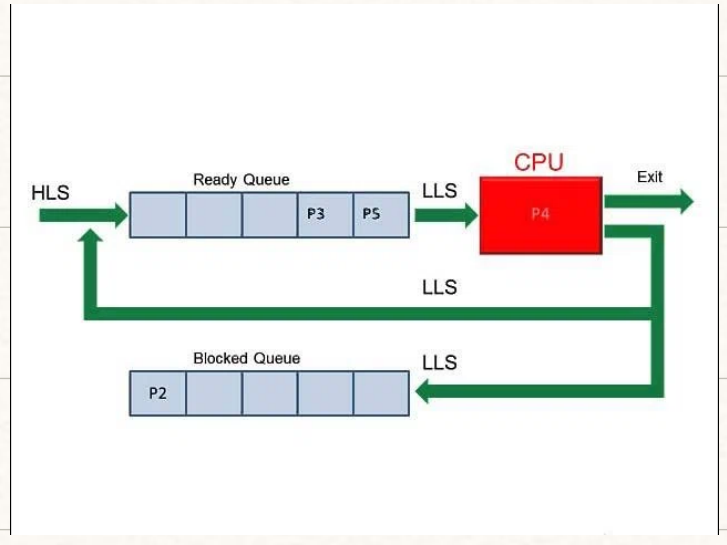
Many systems need to manage data in FIFO order

WHY STUDY THE QUEUE ADT?

Many systems need to manage data in FIFO order

Fundamental for "fair" resource sharing.





How to implement the queue ADT?

The Array Queue

```
class ArrayQueue:
    def __init__(self):
        self._A = []
    def enqueue(self, x):   self._A.append(x)
    def dequeue(self):      return self._A.pop(0)
```

$\Theta(1)$

amortized

$\Theta(n)$

The Array Queue

```
class ArrayQueue:
    def __init__(self):
        self._A = []
    def enqueue(self, x):   self._A.append(x)
    def dequeue(self):      return self._A.pop(0)
```

Time Complexity?

THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

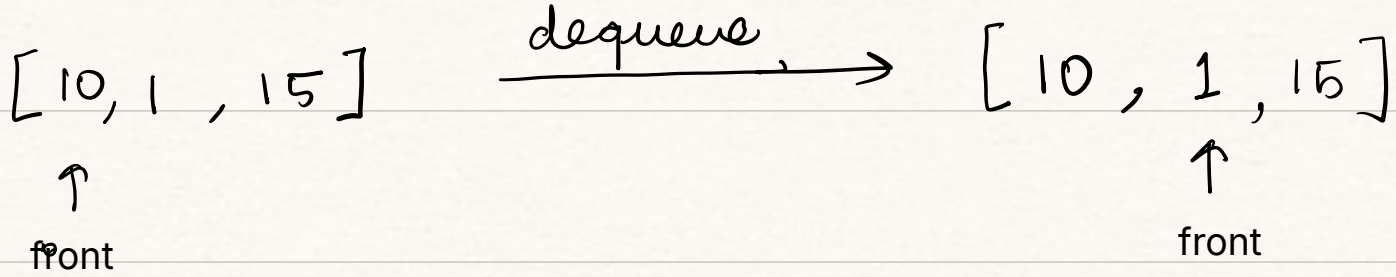
Key idea : Store the index of the first element.

Move the index to right in case of dequeue

THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

Key idea : Store the index of the first element.

Move the index to right in case of dequeue



Time complexity : $\Theta(1)$

Space requirement : $\Theta(m)$

m enqueue operations

n dequeue ops

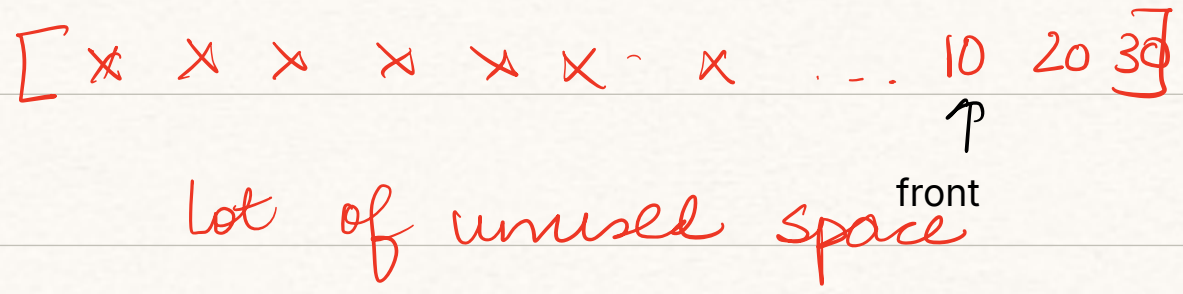
THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

Key idea : Store the index of the first element.

Move the index to right in case of dequeue

ISSUE : A modest size queue will require large memory

Example : wait-list for a restaurant



Lot of unused space

Can we reuse it ?

THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

Key idea : Store the index of the first element.

Move the index to right in case of dequeue

ISSUE : A modest size queue will require large memory

Example : wait-list for a restaurant

Solution : use array in circular fashion

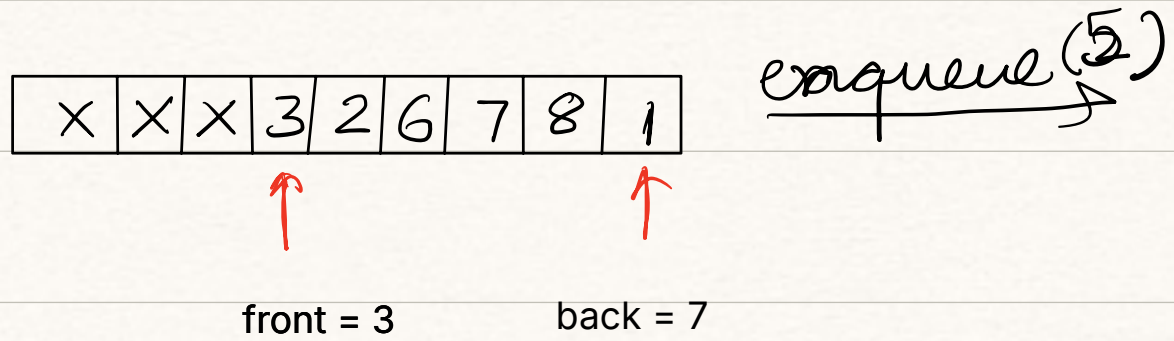
THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

Key idea : Use an array circularly

(1). Create an array of size N

(2). Store the front and end indices of the queue

(3). Update these indices modulo N



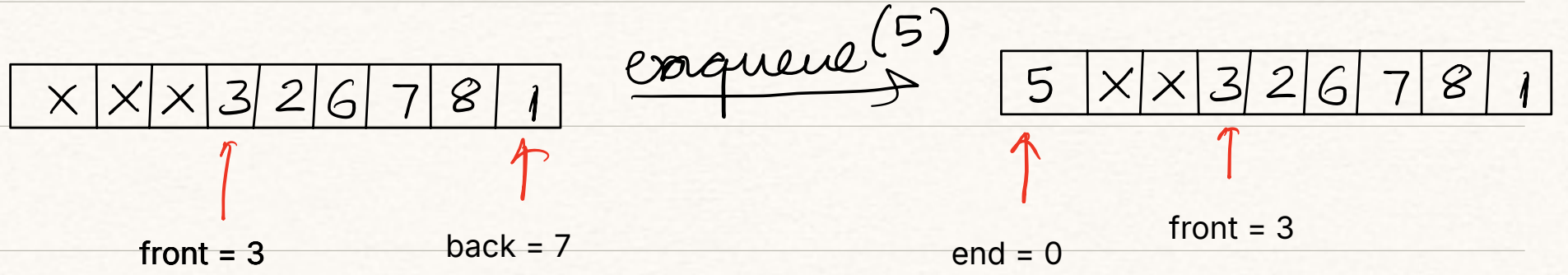
THE ARRAY QUEUE : CAN WE MAKE IT FASTER ?

Key idea : Use an array circularly

(1). Create an array of size N

(2). Store the front and end indices of the queue

(3). Update these indices modulo N

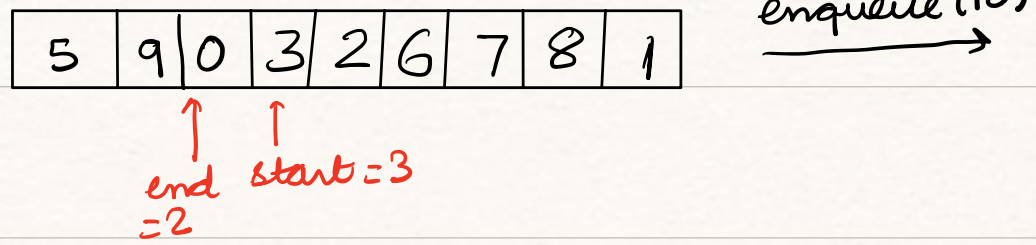


Space requirement: O(N)

Running time: enqueue: O(1),
dequeue: O(1)

THE CIRCULAR ARRAY QUEUE

What to do when array has reached its capacity and we need to enqueue new elements.



→ resize the array when size of queue reaches the capacity

→ By how much

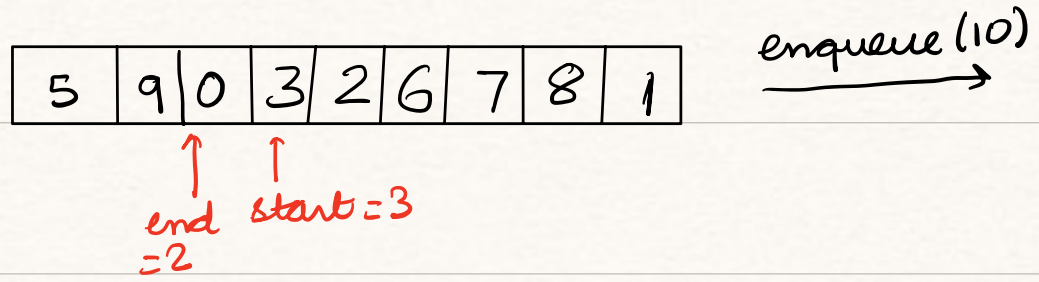
Option 1: by constant amount (+c)

Option 2: by constant factor (×c) ✓

(constant operations amortized)

THE CIRCULAR ARRAY QUEUE

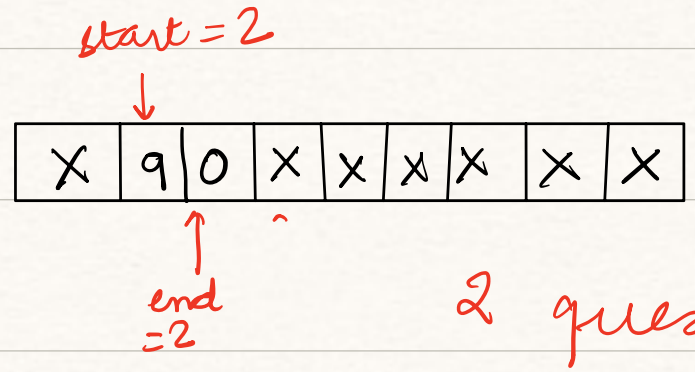
What to do when array has reached its capacity and we need to enqueue new elements.



Idea: create an array of double the current size and copy all elements to this new array

THE CIRCULAR ARRAY QUEUE

What to do when the array has a lot of empty elements.



→ shrink the array

2 questions

① When to shrink?

② By how much?

n: queue size N: array capacity

Option 1: shrink if $n \leq \frac{1}{2}N$

$N \leftarrow \frac{1}{2}N$

→ can oscillate b/w grow and shrink

option 2: shrink if $n \leq \frac{1}{4}N$

$N = \frac{1}{2}N$

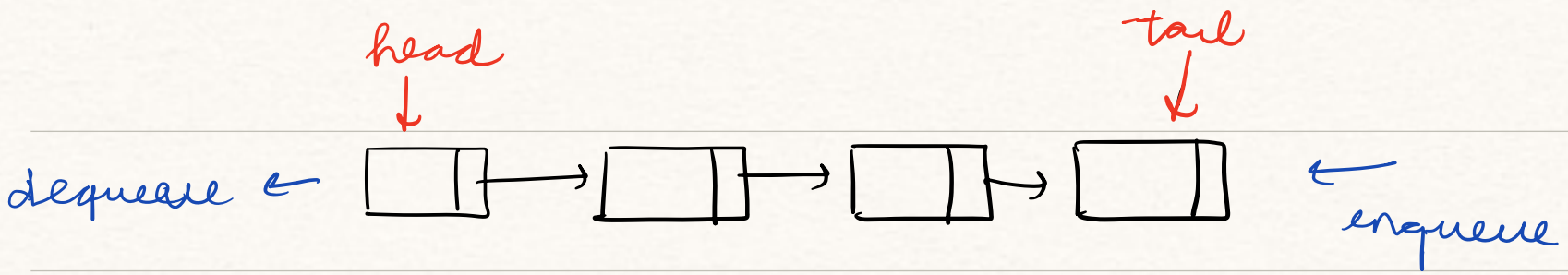
Ensures as much space after every shrink

THE CIRCULAR ARRAY QUEUE

enqueue

dequeue

THE LINKED QUEUE



enqueue(x) dequeue()

```
class LinkedQueue:
    def enqueue(self, x):                    # at the cached tail
        node = self._Node(x, None)
        if self._tail is None: self._head = node
        else:                  self._tail.next = node
        self._tail = node

    def dequeue(self):                       # from the head
        node = self._head
        self._head = node.next
        if self._head is None: self._tail = None
        return node.value
```

Note: you also need to define the constructor (__init__) and Node class

TIME COMPLEXITY

Operation	Naive array	Circular array	Linked (Head+Tail)
enqueue	$\Theta(1)^*$	$\Theta(1)^*$	$\Theta(1)$
dequeue	$\Theta(n)$	$\Theta(1)^*$	$\Theta(1)$
size	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

* amortized

Side note: deque in python provides circular array-based queue.